Modern Reinforcement Learning: From Agents to LLMs

Policy Gradients & Actor–Critic

Lecture 4

Keywords: Policy Gradient Theorem, Score Function Trick, REINFORCE, Actor–Critic, TD Error / Advantage, GAE, Trust-Region Methods, TRPO, PPO

Ilija Bogunovic

Today's Roadmap

1) Motivation & Recap – Why optimize policies directly? From value-based control to parameterized stochastic policies

2) Foundations of Policy Gradients – Score function trick, trajectory gradients

3) Policy Gradient Theorem – Statement, full proof, implications

4) REINFORCE & Variance Reduction – Baselines, advantage, bootstrapping

5) Actor–Critic Methods – TD error, learned critics, bias–variance tradeoffs

6) Advantage Estimation – GAE(λ) and practical estimators

7) Trust-Region & Proximal Methods – TRPO and PPO

📘 Side note / "Keep in mind" box:
Every section introduces a building block that leads to PPO. Advantage estimation + trust-region updates form the foundation of all modern policy optimization algorithms.

Key Concepts

- Direct optimization of stochastic policies
- Gradient of expected return
- Actor–Critic as variance-reduced policy gradient
- Foundations of TRPO → PPO

Why Policy Optimization? (Motivation & Recap)

Why Optimize Policies Directly?

- Value-based control struggles in high-dimensional or continuous action spaces
- Q-learning instability with nonlinear function approximation
- Exploration requires stochasticity → directly parameterized stochastic π_θ is natural
- Optimizing π_θ via $\nabla J(\theta)$ avoids the need to compute argmax_a Q(s,a)
- Policy search methods scale to robotics, continuous control, and LLMs

Policy-Based vs Value-Based Methods

Value-Based RL:

- Learn Q(s,a) or V(s)
- Derive policy via greedy / ε-greedy / softmax(Q)
- Improvement step requires argmax_a Q(s,a)
- Hard in continuous actions; unstable with function approximation

Policy-Based RL:

- Directly parameterize $\pi_\theta(a \mid s)$
- Optimize performance objective $J(\theta)$
- Naturally supports **continuous actions** and **stochastic exploration**
- Updates are **smooth** ($\nabla_\theta \log \pi_\theta(a \mid s)$) rather than discontinuous argmax shifts

Objective: Expected Return $J(\theta)$

Performance Objective

We optimize the policy by maximizing the expected return:

with trajectory return

Trajectory density under the policy:

Trajectory Distribution Under a Policy

- A full trajectory is:

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$$

- The probability of a trajectory under policy π_θ and environment dynamics P:

$$p_\theta(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

Here:

- $\rho(s_0)$: start-state distribution
- $\pi_\theta(a_t|s_t)$: **only part depending on θ**
- $P(s_{t+1}|s_t, a_t)$: environment transitions
- T: episode horizon

Policy Gradient Foundations

Where Does θ Appear in RL?

Recall the trajectory distribution:

$$p_\theta(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

Key observation: θ appears only in $\pi_\theta(a_t \mid s_t)$

Environment terms: $\rho(s_0)$ and $P(s_{t+1} \mid s_t, a_t)$ are independent of θ

Where Does θ Appear in RL?

Recall the trajectory distribution:

$$p_\theta(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

Key observation: θ appears only in $\pi_\theta(a_t \mid s_t)$

Environment terms: $\rho(s_0)$ and $P(s_{t+1} \mid s_t, a_t)$ are independent of θ

Thus

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

(Next slides derive this formally.)

Log-Prob Factorization

Log of the Trajectory Probability

Recall:

$$p_\theta(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid a_t, s_t)$$

📘 Side note:
All environment terms drop because they do not contain θ.

Score Function Trick

Log-Derivative (Score Function) Trick

- For any differentiable probability density $p_\theta(x)$:

$$\nabla p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x)$$

Thus, for any expectation:

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \mathbb{E}_{x \sim p_\theta}[f(x) \nabla_\theta \log p_\theta(x)]$$

This identity enables gradient estimation **without differentiating the environment or sampling process.**

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \mathbb{E}_{x \sim p_\theta}[f(x) \nabla_\theta \log p_\theta(x)]$$

∇J(θ) as an Expectation

Starting from:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)]$$

differentiate (using **the score function trick**)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)\, \nabla_\theta \log p_\theta(\tau)]$$

Using the log-prob factorization:

$$\nabla_\theta \log p_\theta(x) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Plug in:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]}$$

📘 Side note:
This is the core structure of policy gradient estimators.

∇J(θ) as an Expectation

$$\nabla_\theta \mathbb{E}_{x\sim p_\theta}[f(x)] = \mathbb{E}_{x\sim p_\theta}[f(x)\,\nabla_\theta \log p_\theta(x)]$$

Starting from:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$$

differentiate (using **the score function trick**)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)\,\nabla_\theta \log p_\theta(\tau)]$$

Using the log-prob factorization:

$$\nabla_\theta \log p_\theta(x) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Plug in:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[R(\tau)\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

📘 **Side note:**
This is the core structure of policy gradient estimators.

Return-Weighted Log-Gradients

Gradient in Terms of Log-Probabilities

We have:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

Rewriting:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T-1} R(\tau)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]}$$

This is the **return-weighted log-gradient** form — the backbone of REINFORCE.

Monte Carlo Policy Gradient Estimator

Monte Carlo Estimator of ∇J(θ)

From the return-weighted log-gradient form:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T-1} R(\tau)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

Using N sampled trajectories $\tau^{(i)}$:

$$\nabla_\theta \hat{J}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left[\sum_{t=0}^{T-1} R(\tau^{(i)})\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

This is an **unbiased** estimator of the policy gradient.

Policy Gradient Theorem

Policy Gradient Theorem (Statement)

- For any differentiable stochastic policy $\pi_\theta(a \mid s)$

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) Q^\pi(s, a)]}$$

Where

- $d^\pi(s)$: state visitation distribution
- $Q^\pi(s, a)$: action-value function under policy π

📘 Side note:
Only the policy appears in the gradient;
transition dynamics vanish entirely.

Policy Gradient Theorem: Proof (Part I)

We have shown previously:

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[\sum_{t=0}^{T-1} R(\tau)\,\nabla_\theta \log \pi_\theta(a_t \,|\, s_t)\Big]$$

Rewrite the return in terms of return-to-go:

$$R(\tau) = G_t + \sum_{k=0}^{t-1} \gamma^k r_k$$

but the **past** rewards do not depend on a_t , so:

$$\mathbb{E}[R(\tau)\,\nabla_\theta \log \pi_\theta(a_t \,|\, s_t)] = \mathbb{E}[G_t\,\nabla_\theta \log \pi_\theta(a_t \,|\, s_t)]$$

Thus: $\nabla_\theta J(\theta) = \mathbb{E}\Big[\sum_{t=0}^{T-1} G_t\,\nabla_\theta \log \pi_\theta(a_t \,|\, s_t)\Big]$

Policy Gradient Theorem: Proof (Part II)

From Return-to-Go to Q-function

We reached:

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[\sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(a_t \,|\, s_t)\Big]$$

Now use:

$$\mathbb{E}[G_t \,|\, s_t = s, a_t = a] = Q^\pi(s, a)$$

Therefore, taking expectation over s_t, a_t

$$\mathbb{E}[G_t \nabla_\theta \log \pi_\theta(a_t \,|\, s_t)] = \mathbb{E}[Q^\pi(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t \,|\, s_t)]$$

Summing over t with the discounted state visitation distribution $d^\pi(s)$:
$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \,|\, s) Q^\pi(s, a)]$

Interpreting the Policy Gradient Theorem

Policy Gradient Theorem (recap)

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}\left[Q^\pi(s,a)\,\nabla_\theta \log \pi_\theta(a\,|\,s)\right]$$

What This Expression Means

- Actions that lead to higher return should become more likely.
  - if $Q^\pi(s,a)$ is large —> increase $\log \pi_\theta(a\,|\,s)$
  - if $Q^\pi(s,a)$ is small —> decrease it
- Credit assignment becomes “local”:
  - Only the immediate state-action pair (s,a) affects the gradient.

Environment dynamics do not appear: gradient only depends on π

Expectation over the visitation distribution: $d^\pi(s) =$ how often we visit state s under policy π

The gradient follows the trajectory distribution of the current policy.

Implications of the Policy Gradient Theorem 1/2

1. Model-Free Policy Optimization $\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}[\nabla_\theta Q^\pi(s,a)\ \log \pi_\theta(a\,|\,s)]$

The gradient depends only on π_θ, not on dynamics $P(s'\,|\,s,a)$

Thus: 1) No environment model needed; 2) Works in unknown, stochastic, non-differentiable environments; 3) Applicable to simulators, robotics, games, LLMs.

Implications of the Policy Gradient Theorem 1/2

1. Model-Free Policy Optimization $\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}[\nabla_\theta Q^\pi(s,a)\ \log \pi_\theta(a\,|\,s)]$

The gradient depends only on π_θ, not on dynamics $P(s'\,|\,s,a)$

Thus: 1) No environment model needed; 2) Works in unknown, stochastic, non-differentiable environments; 3) Applicable to simulators, robotics, games, LLMs.

2. Variance Reduction Becomes Essential

Replacing $R(\tau)$ with the local value $Q^\pi(s,a)$ drastically reduces variance

Encourages baselines, critics, and advantage estimation

Implications of the Policy Gradient Theorem 1/2

1. Model-Free Policy Optimization $\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}[\nabla_\theta Q^\pi(s,a)\ \log \pi_\theta(a\,|\,s)]$

The gradient depends only on π_θ, not on dynamics $P(s'\,|\,s,a)$

Thus: 1) No environment model needed; 2) Works in unknown, stochastic, non-differentiable environments; 3) Applicable to simulators, robotics, games, LLMs.

2. Variance Reduction Becomes Essential

Replacing $R(\tau)$ with the local value $Q^\pi(s,a)$ drastically reduces variance

Encourages baselines, critics, and advantage estimation

3. Learning Requires Good Estimates of Q

Exact Q^π is rarely known. In practice, we approximate Q via: 1) Monte Carlo return (REINFORCE) 2) Bootstrapped critics (Actor–Critic) 3) GAE for stability in deep RL

Implications of the Policy Gradient Theorem 2/2

4. Exploration–Exploitation Tradeoff Is Built-In

- Stochastic policy gradient ensures exploration.

5. Foundation for Modern Algorithms

This theorem underlies:

- REINFORCE

- Actor–Critic / Advantage Actor–Critic (A2C/A3C)

- TRPO / PPO

- Modern policy optimization for LLMs (PPO-KL, variants)

Every method is essentially:

$$\text{Update } \theta \propto \hat{A}\, \nabla_\theta \log \pi_\theta$$

REINFORCE & Variance Reduction

REINFORCE: Monte Carlo Policy Gradient

REINFORCE Update Rule

Using the Monte Carlo return-to-go G_t

$$\nabla_\theta J(\theta) \approx \sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Gradient-ascent update:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

REINFORCE is Unbiased

- **Key Property**

$$\mathbb{E}\left[\sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] = \nabla_\theta J(\theta)$$

REINFORCE exactly matches the true policy gradient in expectation.

- **Why It Is Unbiased**

1. G_t is an unbiased estimator of $Q^\pi(s_t, a_t)$: $\mathbb{E}[G_t \mid s_t, a_t] = Q^\pi(s_t, a_t)$
2. The policy gradient theorem says: $\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^\pi(s, a)]$
3. Substitute 1 into 2 → unbiasedness.

- **Implication:** REINFORCE will, in expectation, move θ in exactly the correct ascent direction.

Variance Problem in REINFORCE 1/2

Why Variance Is a Major Issue

— REINFORCE uses the Monte Carlo return-to-go:

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

— This includes **all future randomness**, so:

- Rewards far in the future strongly affect early updates
- High variability across episodes
- Long horizons → variance grows dramatically
- Stochastic policies amplify fluctuations
- Sparse rewards → almost no meaningful signal

Variance Problem in REINFORCE 2/2

Consequences:

- Very noisy gradient estimates
- Slow or unstable learning
- Requires small learning rates
- High sample complexity

Key Insight: The gradient is **correct on average**, but each sample is **extremely noisy**:

$$\nabla_\theta \hat{J}(\theta) = m \pm \text{(huge variance)}$$

We need variance reduction — baselines, critics, and advantages.

Baselines (Unbiased Variance Reduction)

Subtracting a Baseline

- For any function $b(s_t)$ that does not depend on the action a_t :

$$\mathbb{E}\left[(G_t - b(s_t))\,\nabla_\theta \log \pi_\theta(a \mid s)\right] = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(a \mid s)\right]$$

Thus, the gradient remains unbiased.

Baselines (Unbiased Variance Reduction)

Subtracting a Baseline

- For any function $b(s_t)$ that does not depend on the action a_t :

$$\mathbb{E}\left[(G_t - b(s_t))\,\nabla_\theta \log \pi_\theta(a \mid s)\right] = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(a \mid s)\right]$$

Thus, the gradient remains unbiased.

- Intuition:

  - Baseline shifts the reward signal up or down

  - But since it does not depend on the action, the expected gradient is unchanged

  - However, the variance can drop dramatically

  - Baseline = "expected return from this state" → makes learning more stable.

- Optimal Baseline (in theory): $b^*(s) = V^\pi(s)$. This choice minimizes variance of the estimator.

Advantage Function

From Baselines to Advantage

- Using a baseline $b(s_t)$:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[(G_t - b(s_t))\,\nabla_\theta \log \pi_\theta(a \mid s)\right]$$

- Choosing the best baseline (in theory):

$$b(s_t) = V^\pi(s_t)$$

gives the advantage function:

$$\boxed{A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)}$$

Gradient with Advantage: $\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[A^\pi(s_t, a_t)\,\nabla_\theta \log \pi_\theta(a \mid s)\right]}$

This is the preferred form in modern policy-gradient algorithms.

📘 Why Advantage?

- Measures how much better an action is compared to average actions at that state
- Reduces variance without changing the expectation of the gradient
- Zero-mean signal around each state → more stable learning

Bootstrapping: Reducing Variance Further 1/2

Bootstrapping: Reducing Variance Further

- Monte Carlo (MC) uses full return:

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

→ unbiased but **high variance**

Bootstrapping uses an estimate of future value:

$$G_t \approx r_t + \gamma V^{\pi}(s_{t+1})$$

→ slightly biased but much lower variance

Bootstrapping: Reducing Variance Further 2/2

TD Error as a One-Step Bootstrap

$$\delta_t = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$$

This TD error is a **low-variance proxy for the advantage:**

$$A^\pi(s_t, a_t) \approx \delta_t$$

- Why bootstrapping helps:

- Reduces dependence on long future rewards
- Provides more immediate learning signal
- Makes learning stable even in long-horizon tasks
- Prepares the ground for Actor–Critic methods

Actor-Critic

Actor–Critic: Motivation

Why Actor–Critic?

- We want an estimator of the policy gradient: $\nabla_\theta J(\theta) = \mathbb{E}\left[A^\pi(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$ but:
  - $Q^\pi(s, a)$ is unknown
  - $V^\pi(s)$ is unknown
  - MC returns G_t have high variance
  - TD returns have lower variance but introduce bias

Actor–Critic: Motivation

Why Actor–Critic?

- We want an estimator of the policy gradient: $\nabla_\theta J(\theta) = \mathbb{E}\left[A^\pi(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$ but:
  - $Q^\pi(s, a)$ is unknown
  - $V^\pi(s)$ is unknown
  - MC returns G_t have high variance
  - TD returns have lower variance but introduce bias

Idea of Actor–Critic: Split the model into two components

- **Actor:** the policy $\pi_\theta(a \mid s)$
  - Learns how to act
- **Critic:** value function $V_\phi(s)$
  - Learns how good the state is

📘 **Actor–Critic Update Structure**

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

where $\hat{A}_t$ is supplied by the critic.

The critic provides a low-variance estimate of the advantage, enabling stable updates.

Actor–Critic: Motivation

Why Actor–Critic?

- We want an estimator of the policy gradient: $\nabla_\theta J(\theta) = \mathbb{E}\left[A^\pi(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$ but:
  - $Q^\pi(s, a)$ is unknown
  - $V^\pi(s)$ is unknown
  - MC returns G_t have high variance
  - TD returns have lower variance but introduce bias

Idea of Actor–Critic: Split the model into two components

- **Actor:** the policy $\pi_\theta(a \mid s)$
  - Learns how to act
- **Critic:** value function $V_\phi(s)$
  - Learns how good the state is

📘 **Actor–Critic Update Structure**

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

where $\hat{A}_t$ is supplied by the critic.

The critic provides a low-variance estimate of the advantage, enabling stable updates.

TD Learning for the Critic

Learning the Value Function

- The critic aims to approximate: $V^{\pi}(s) = \mathbb{E}[G_t \mid s_t = s]$

- We train a parametric value function using Temporal-Difference learning: $V_{\phi}(s_t) \leftarrow V_{\phi}(s_t) + \beta\delta_t$

where the TD error is:

$$\boxed{\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)}$$

📘 **Why TD Learning?**

- Uses one-step bootstrapping
- Much lower variance than Monte Carlo returns
- Enables online/semi-online updates
- Provides fast, stable learning signals for the actor

TD Learning for the Critic

Learning the Value Function

- The critic aims to approximate: $V^{\pi}(s) = \mathbb{E}[G_t \mid s_t = s]$

- We train a parametric value function using Temporal-Difference learning: $V_{\phi}(s_t) \leftarrow V_{\phi}(s_t) + \beta\delta_t$

where the TD error is:

$$\boxed{\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)}$$

Critic Loss Function

- A common formulation:

$$L(\phi) = \frac{1}{2}\left(r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)\right)^2$$

$$\nabla_{\phi} = -\delta_t \nabla_{\phi} V_{\phi}(s_t)$$

📘 **Why TD Learning?**

- Uses one-step bootstrapping
- Much lower variance than Monte Carlo returns
- Enables online/semi-online updates
- Provides fast, stable learning signals for the actor

TD Error as an Advantage Estimate

TD Error

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

The TD error measures how much better (or worse) the taken action a_t performed relative to the critic's expectation.

TD Error ≈ Advantage

In expectation under the policy: $\mathbb{E}[\delta_t \mid s_t, a_t] = Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)$

Thus: $\boxed{A^\pi(s_t, a_t) \approx \delta_t}$

📘 Why This Matters?

- TD error provides a low-variance, online, incremental estimate of the advantage.
- Enables learning at every time step, not only after full episodes.
- Forms the backbone of Actor–Critic algorithms.

Actor Update Using Estimated Advantage

Actor Gradient Update

- Once we have an advantage estimate $\hat{A}_t$:

$$\nabla_\theta J(\theta) \approx \hat{A}_t(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

- Stochastic gradient ascent update:

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

where: $\hat{A}_t = \delta_t$ (one-step TD advantage), or $\hat{A}_t = G_t - V_\phi(s_t)$ (MC advantage), or $\hat{A}_t$ from GAE (later)

Actor Update Using Estimated Advantage

Actor Gradient Update

- Once we have an advantage estimate $\hat{A}_t$:

$$\nabla_\theta J(\theta) \approx \hat{A}_t(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

- Stochastic gradient ascent update:

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

where: $\hat{A}_t = \delta_t$ (one-step TD advantage), or $\hat{A}_t = G_t - V_\phi(s_t)$ (MC advantage), or $\hat{A}_t$ from GAE (later)

Meaning of the Update

- if $\hat{A}_t > 0$ increase log-probability of the chosen action
- if $\hat{A}_t < 0$ decrease log-probability
- if $\hat{A}_t = 0$ no update

Unified Actor–Critic Formula: $$\theta \leftarrow \theta + \alpha \sum_t \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

This is the core update used in all modern On-Policy PG algorithms.

Bias–Variance Tradeoff in Actor–Critic

- **Sources of Variance**
  - Monte Carlo returns G_t
  - Long trajectories
  - Stochastic rewards
  - Stochastic policies
  - Sparse or delayed rewards
- High variance makes REINFORCE unstable.

- **Sources of Bias**
  - Bootstrapped value estimates $V_\phi(s)$
  - Function approximation errors in the critic
  - Imperfect advantage estimates
  - Finite-sample training of the critic
- Small bias is often acceptable if variance is reduced substantially.

Method	Bias	Variance
Monte Carlo	none	very high
TD(0)	some	low
TD(λ), GAE(λ)	adjustable	adjustable

Actor–Critic dynamically balances **bias (from critic)** vs **variance (from MC)**.

Actor–Critic: Pseudocode

- On-Policy Actor–Critic Algorithm

Initialise

Policy parameters θ · Value function parameters ϕ

repeat

Generate trajectory $(s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$

for $t = 0$ to $T-1$

Critic update (value function)

$$\delta_t = r_t + \gamma \cdot V_\phi(s_{t+1}) - V_\phi(s_t)$$

$$\phi \leftarrow \phi + \beta \cdot \delta_t \cdot \nabla_\phi V_\phi(s_t)$$

TD error δ_t shared

Actor update (policy)

$$\theta \leftarrow \theta + \alpha \cdot \delta_t \cdot \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

until convergence

Natural Gradient Motivation 1/2

Problem with Vanilla Policy Gradient

Standard update:

$$\theta \leftarrow \theta + \alpha g, \qquad g = \mathbb{E}[A^{\pi}(s,a)\,\nabla_{\theta}\log \pi_{\theta}(a\,|\,s)]$$

This uses **Euclidean geometry** in parameter space.

But:

- A small step in θ may produce a large change in the policy
- A large step in θ may produce a small change in the policy
- Learning becomes unstable and sensitive to parameterization

Natural Gradient Motivation 2/2

Natural Policy Gradient Idea

Use the geometry of **policy space**, not parameter space.

Define the **Fisher Information Matrix:**

$$F = \mathbb{E}[\nabla_\theta \log_\theta \pi_\theta(a \mid s) \nabla_\theta \log_\theta \pi_\theta(a \mid s)^T]$$

Natural gradient direction

$$\boxed{\tilde{g} = F^{-1} g}$$

This accounts for how changes in θ affect the distribution over actions

📘 Some Intuition

- Natural gradient moves the policy the same amount in distribution space regardless of parameterization
- Leads to more stable and efficient learning
- Forms the basis of TRPO and motivates the surrogate/ KL constraints of PPO

Advantage Estimation

Problems With Simple Advantage Estimators

Problems With Simple Advantage Estimators

1) **Monte Carlo Advantage**

$$\hat{A}_t = G_t$$

- Unbiased
- Extremely high variance, especially for long horizons
- Leads to unstable actor updates

2) **TD(0) Advantage**

$$\hat{A}_t = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

- Low variance
- But biased, depends heavily on critic accuracy
- Short-term signal only (no multi-step credit)

Problems With Simple Advantage Estimators

Problems With Simple Advantage Estimators

1) **Monte Carlo Advantage**

$$\hat{A}_t = G_t$$

- Unbiased
- Extremely high variance, especially for long horizons
- Leads to unstable actor updates

2) **TD(0) Advantage**

$$\hat{A}_t = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

- Low variance
- But biased, depends heavily on critic accuracy
- Short-term signal only (no multi-step credit)

Motivation for GAE (Generalized Advantage Estimation) [Schulman et al. (2015)]

GAE(λ) blends multiple n-step returns:

- Helps reduce variance
- Controls bias smoothly via parameter λ
- Provides a long-term but stable advantage estimator
- Dramatically improves PPO/A2C performance

Generalized Advantage Estimator

- TD Residuals (1-step Advantages): $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- GAE(λ) averages multi-step advantages with exponentially decaying weights:

$$\boxed{\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}}$$

📘 **Intuition**

This combines:
- TD(0) (when λ = 0)
- Monte Carlo advantages (when λ = 1)

Equivalent Truncated Form (finite horizon):

$$\hat{A}_t = \delta_t + \gamma\lambda \, \delta_{t+1} + \gamma^2\lambda^2 \, \delta_{t+2} + \cdots + \gamma^{T-t-1}\lambda^{T-t-1} \, \delta_{T-1} \, .$$

Generalized Advantage Estimator

- TD Residuals (1-step Advantages): $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- GAE(λ) averages multi-step advantages with exponentially decaying weights:

$$\boxed{\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}}$$

📘 **Intuition**

This combines:
- TD(0) (when λ = 0)
- Monte Carlo advantages (when λ = 1)

Equivalent Truncated Form (finite horizon):

$$\hat{A}_t = \delta_t + \gamma\lambda \, \delta_{t+1} + \gamma^2\lambda^2 \, \delta_{t+2} + \cdots + \gamma^{T-t-1}\lambda^{T-t-1} \, \delta_{T-1} \, .$$

Key Idea — GAE produces an advantage estimator that is:

- Low variance (due to λ < 1)
- Low bias (due to multi-step accumulation)
- Smooth (avoids TD jumps and MC noise)

GAE Bias–Variance Tradeoff

- GAE(λ) Blends Multi-Step Returns:

$$\hat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

λ controls how much future information is included in the advantage estimate.

Intuition:

- Small λ:
    - Critic dominates
    - Fast, stable learning
    - More bias due to reliance on V_ϕ
- Large λ:
    - More Monte Carlo structure
    - Lower bias, but increased variance
    - Sensitive to noise and reward sparsity

📘 **Why GAE Works Well:**

- **Smooths the variance spikes from MC returns**
- **Maintains long-horizon credit assignment**
- **Robust to critic imperfections**
- **Empirically optimal λ is typically 0.90–0.97 (Atari, MuJoCo, PPO, LLM PPO-KL/GRPO)**

Modern Policy Optimization (Trust Regions, TRPO, PPO)